

Ahmet Kazankaya

AI-ASSISTED DEVELOPER / AUTOMATION ENGINEER

One human, several agents. I decide, review and debug; they type.

Istanbul, Turkey (remote) · ahmetkazankaya1@gmail.com · +90 536 214 8114 · linkedin.com/in/ahmetkazankaya · github.com/kasparovabi

SUMMARY

I came to software through production work rather than a computer science degree, and my focus is now fully on systems and automation. Since early 2026 I have shipped, with Claude Code as the primary tool, a task system that has been running for four months at a foundation, an automation line that lists and prices real products on Etsy, an ESP32 desktop robot with voice, and two open source skill sets other people install into their own agent setups. A model writes the code and the failures land on me, so I know where these tools mislead; the cases below are what that knowledge looks like.

HOW I WORK WITH AI

Working model. One human, several agents. I write the architecture and the constraints into per repository steering files, route each task to the cheapest model that can do it, and read every diff before it lands. Because the automation carries the repetitive load, I run several of these lines at the same time rather than one after another.

Tools. Claude Code across a Windows and a macOS machine joined over Tailscale, so one session works on both. Claude Agent SDK inside my own applications. Codex CLI, Cursor, Antigravity and Gemini CLI alongside it; Ollama and OpenRouter for model routing; MCP servers for Blender, NotebookLM and my own tools. Verification runs headless: Unity batchmode builds, xcodebuild with simulator screenshots, pytest and vitest suites the agent has to pass before a task is done.

Risks and limits. I wrote guard-20 and audit-20 because the same twenty failure classes kept reappearing in generated code. I refuse blacklist style fixes: they patch the layer where the symptom shows, not the one that broke. One research loop I built rejected 87 hypotheses over 170 iterations and returned the answer I did not want: no tradable edge was left in that space.

CASES

Maarif Task Management · Türkiye Maarif Vakfı · in production, running for four months

The director used to read every incoming email himself, decide who should do each job, then track every delivery personally. The system now reads the mailbox, turns each message into a task definition with an LLM, assigns it, and puts the state of every job on an office TV board.

Symptom: generated documents drifted from the institution's templates, and a text diff showed nothing.

Root cause: the drift was visual, so the thing I was checking could not see it.

Fix: compare delivered files against fixed templates visually, which made template stability measurable; move the watcher off raw mail to an LLM written job definition.

Stack: Python, FastAPI, SQLAlchemy, SQLite, Next.js, Swift, Microsoft Graph, Outlook 365, Telegram.

Etsy Factory and Demand Engine · own production business · running daily

Public domain museum works and locally generated art flow into Printify products and Etsy listings, through a Telegram approval gate, on a schedule, driven from my phone over the tailnet. A pricing engine pulls Printify's own shipping table per variant and country and disables combinations that cannot be profitable.

Symptom: the scout agents kept producing near duplicate product ideas.

First guess: the model. Changing it changed nothing.

Root cause: the shape of the data handed to the agents.

Fix: restructure the input. The clearest case I have of an AI failure that is not an AI problem.

Stack: Python, Swift, Etsy API, Printify, Telegram, launchd, Tailscale.

Rıfkı, a desktop robot · ESP32-S3 · working prototype

Microphone to speech recognition to an LLM to speech synthesis to head and arm servos, with wake word detection, over the air firmware updates and camera based face tracking.

Symptom: on battery the board browned out and boot looped, for a month.

First guess: servos stay passive during boot, so they cannot be the cause. Wrong.

Signal: watching the hardware, the servos twitch the instant the battery goes in.

Root cause: at reset the PWM pins float; three SG90 servos each pull about 300 mA of inrush, and with the WiFi power peak they collapse the boost rail.

Fix: drive those pins to OUTPUT LOW with pulldown in the first line of the board constructor. No hardware change; the screen came up on the first try.

Second fault, same machine: the speech pipeline invented a phrase out of silence. Filtering the phrase would have patched the wrong layer. Torch's C++ file open corrupts the Turkish character in the project path, so the voice activity detector raised on every call and a broad except turned that into a warning; with it dead, endpointing never fired and the transcriber got silence. Copying the model to an ASCII cache path fixed it.

Stack: C++, PlatformIO, ESP-IDF, Python, Deepgram, Groq, ElevenLabs.

guard-20, audit-20 and Mythos Scaffold · open source · github.com/kasparovabi

Twenty recurring AI coding failure classes turned into enforceable process. guard-20 states them as write time rules; audit-20 runs a twenty pass verification gate whose final pass exists only to falsify the findings of the previous nineteen, so the aggressive passes do not bury you in false positives.

Symptom: the scaffold measurably degraded output on the newest model generation, the opposite of its purpose.

Root cause: over scaffolding. The newer the model, the more the extra process costs it.

Fix: model gating, the ceiling written into the documentation, and an eval harness that runs bare and scaffolded arms so the claim is measured rather than assumed.

Cebimde Claude · building iOS apps from the phone, without sitting at a Mac

Making an iOS app normally means a Mac with Xcode open in front of you. This turns the Mac into a build server you never look at: you describe the app from your phone, the agent scaffolds an XcodeGen SwiftUI project, compiles it, and sends back a simulator screenshot; a second command signs it and installs it on the phone in your hand. Failed builds go back to the agent, which fixes them and rebuilds unprompted.

Symptom: every dropped connection left a ghost, writing output to a dead socket while a new agent started on the same project.

Root cause: no identity on the connection, no ownership rule per project.

Fix: a connection epoch, a stop rule for hanging agents, and session ids persisted to disk so a conversation survives the app being killed.

STACK

Everything below appears in code I have shipped.

Languages: Python, TypeScript, JavaScript, Swift, C++ (ESP32/Arduino), C#, SQL, MQL5, Shell

Backend and data: FastAPI, SQLAlchemy, SQLite, PostgreSQL, Prisma, Supabase, NestJS, Redis

Frontend and apps: Next.js, React, Tailwind, Vite, Turborepo, SwiftUI, Capacitor, Tauri, Expo, PySide6

Embedded: ESP-IDF, PlatformIO, Arduino, servo and audio hardware bring up

Infrastructure: Git, Docker, Vercel, Netlify, Cloudflare Workers, Tailscale, launchd and cron, pm2

Integrations: REST APIs, Microsoft Graph and Outlook 365, Etsy, Printify, Telegram, WhatsApp, Deepgram, ElevenLabs, Groq, KIE.AI, MetaTrader5

Testing: pytest, vitest, Jest, XCTest, Playwright, headless batchmode CI

EXPERIENCE

AI Solutions Designer · Türkiye Maarif Vakfı · Istanbul · Mar 2026 to present · internal systems and automation for the Directorate of Communications

Generative AI Engineer · Pepper Root, Creative AI Studio · remote · Nov 2025 to Jan 2026

AI Artist · Joy Game · Istanbul · Aug 2025 to Oct 2025 · game assets and pipeline automation

Chief GenAI Art Director · GPT Journey · Istanbul · Mar 2024 to Aug 2025

Generative AI Creator · Fiverr · remote · Oct 2022 to Jun 2025

Graphic Designer and Photographer · Creww Digital · Istanbul · Jun 2020 to Nov 2022

Official Photographer · Istanbul Governorship · Istanbul · Jan 2018 to Apr 2020

News Editor · Skala Media Solutions · Istanbul · Feb 2017 to Jan 2018

Teacher · ARGEM · Istanbul · Sep 2016 to Feb 2017

EDUCATION, BACKGROUND AND RESEARCH

I keep a searchable knowledge layer over what I read, with weekly lint runs that flag contradictions and claims gone stale. When a component misbehaves I go to the source before the second attempt rather than after the tenth; on one hardware track that meant 112 documented iterations before I committed to a design. I verify current practice before choosing a stack, because a model's training data is always older than the problem in front of me.

Mechanical Engineering, Namık Kemal University, from 2012; I left before finishing the degree and learned the rest by building. Ten years in visual production before software is where the tooling instinct came from: Stable Diffusion and ComfyUI on my own machines from 2023, node based workflows and LoRA fine tuning treated as production tooling rather than demos. My focus now is software and automation.

LANGUAGES

Turkish (native) · English (fluent)